

[Type here]



AKADEMIA GÓRNICZO-HUTNICZA

im. Stanisława Staszica w Krakowie

**WYDZIAŁ INŻYNIERII
MECHANICZNEJ I ROBOTYKI**

Praca dyplomowa inżynierska

Artur Skowroński

Imię i nazwisko

Automatyka i Robotyka

Kierunek studiów

**Manipulacja modelem ramienia robotycznego z użyciem KINECT
SDK przy pomocy Inventor API**

Temat pracy dyplomowej

Dr inż. Tomasz Buratowski

Promotor pracy

.....

Ocena

Kraków, rok 2012/2013

[Type here]

[Type here]

[Type here]

[Type here]

Kraków, 23.01.2013r.

Imię i nazwisko: Artur Skowroński

Nr albumu: 231405

Kierunek studiów: **Automatyka i Robotyka**

Specjalność:

OŚWIADCZENIE

Świadomy odpowiedzialności karnej za poświadczanie nieprawdy oświadczam, że niniejszą inżynierską pracę dyplomową wykonałem osobiście i samodzielnie oraz nie korzystałem ze źródeł innych niż wymienione w pracy.

Jednocześnie oświadczam, że dokumentacja pracy nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz. U. z 2006 r. Nr 90 poz. 631 z późniejszymi zmianami) oraz dóbr osobistych chronionych prawem cywilnym. Nie zawiera ona również danych i informacji, które uzyskałem w sposób niedozwolony. Wersja dokumentacji dołączona przeze mnie na nośniku elektronicznym jest w pełni zgodna z wydrukiem przedstawionym do recenzji.

Zaświadczam także, że niniejsza inżynierska praca dyplomowa nie była wcześniej podstawą żadnej innej urzędowej procedury związanej z nadawaniem dyplomów wyższej uczelni lub tytułów zawodowych.

.....

podpis dyplomanta

[Type here]

[Type here]

[Type here]

[Type here]

Kraków, 23.01.2012r.

Imię i nazwisko: Artur Skowrońs

Adres korespondencyjny: Kolbuszowa Dolna, Tarnobrzaska 157, 36-100 Kolbuszowa

Temat pracy dyplomowej inżynierskiej: Manipulacja modelem ramienia robotycznego z użyciem KINECT SDK przy pomocy Inventor API.

Rok ukończenia: 2013

Nr albumu: 231405

Kierunek studiów: Automatyka i Robotyka

Profil dyplomowania:

OŚWIADCZENIE

Niniejszym oświadczam, że zachowując moje prawa autorskie, udzielam Akademii Górniczo-Hutniczej im. S. Staszica w Krakowie nieograniczonej w czasie nieodpłatnej licencji niewyłącznej do korzystania z przedstawionej dokumentacji inżynierskiej pracy dyplomowej, w zakresie publicznego udostępniania i rozpowszechniania w wersji drukowanej i elektronicznej¹.

Publikacja ta może nastąpić po ewentualnym zgłoszeniu do ochrony prawnej wynalazków, wzorów użytkowych, wzorów przemysłowych będących wynikiem pracy inżynierskiej².

Kraków,

data podpis dyplomanta

¹ Na podstawie Ustawy z dnia 27 lipca 2005 r. Prawo o szkolnictwie wyższym (Dz.U. 2005 nr 164 poz. 1365) Art. 239. oraz Ustawy z dnia 4 lutego 1994 r. o prawie autorskim i prawach pokrewnych (Dz.U. z 2000 r. Nr 80, poz. 904, z późn. zm.) Art. 15a. "Uczelni w rozumieniu przepisów o szkolnictwie wyższym przysługuje pierwszeństwo w opublikowaniu pracy dyplomowej studenta. Jeżeli uczelnia nie opublikowała pracy dyplomowej w ciągu 6 miesięcy od jej obrony, student, który ją przygotował, może ją opublikować, chyba że praca dyplomowa jest częścią utworu zbiorowego."

² Ustawa z dnia 30 czerwca 2000r. – Prawo własności przemysłowej (Dz.U. z 2003r. Nr 119, poz. 1117 z późniejszymi zmianami) a także rozporządzenie Prezesa Rady Ministrów z dnia 17 września 2001r. w sprawie dokonywania i rozpatrywania zgłoszeń wynalazków i wzorów użytkowych (Dz.U. nr 102 poz. 1119 oraz z 2005r. Nr 109, poz. 910).

[Type here]

[Type here]

[Type here]

[Type here]

Kraków, 23.01.2012r.

AKADEMIA GÓRNICZO-HUTNICZA

WYDZIAŁ INŻYNIERII MECHANICZNEJ I ROBOTYKI

TEMATYKA PRACY DYPLOMOWEJ INŻYNIERSKIEJ

dla studenta IV roku studiów stacjonarnych

.....
Artur Skowroński

imię i nazwisko studenta

TEMAT PRACY DYPLOMOWEJ INŻYNIERSKIEJ:

Manipulacja modelem ramienia robotycznego z użyciem KINECT SDK przy pomocy
Inventor API
.....
.....
.....

Promotor pracy: dr inż. Tomasz Buratowski

Recenzent pracy:

Podpis dziekana:

PLAN PRACY DYPLOMOWEJ:

1. Omówienie tematu pracy i sposobu realizacji z promotorem.
2. Zebranie i opracowanie literatury dotyczącej tematu pracy
3. Opracowanie przykładu programu
4. Opracowanie redakcyjne.

Kraków,
.....

data

podpis dyplomanta

TERMIN ODDANIA DO DZIEKANATU:

.....

.....
podpis promotora

[Type here]

[Type here]

[Type here]

[Type here]

Wstęp

Założenia pracy

Celem pracy jest próba wykorzystania dynamicznie rozwijającego się w ostatnich latach trendu znanego jako Naturalne Interfejsy Użytkownika go na potrzeby sterowania ramieniem robotycznym. W ramach pracy powstanie próba zaadaptowania rozwiązań istniejących na rynku konsumenckim i sprawdzenie możliwości ich wykorzystania podczas kontroli ruchu antropomorficznie zbudowanego ramienia. Powstanie również oprogramowanie, które umożliwi przedstawienie w praktyce efektów analiz i testów. W dalszej perspektywie powstały kod źródłowy będzie mógł stanowić podstawę powstania biblioteki, która w założeniu pozwoli na zaimplementowanie w łatwy sposób współpracy między urządzeniem a kamerą głębi, która tu pełnić będzie rolę systemu wizyjnego o bardziej zaawansowanych możliwościach ze względu na zdolność pomiaru dystansu obserwowanego obiektu od kamery.

Podczas testów wykorzystany zostanie Kinect for Xbox firmy Microsoft. Jest to urządzenie, które pomimo niskiego kosztu ekonomicznego posiada akceptowalne możliwości i pozwala na relatywnie precyzyjny pomiar odległości.

Aby ułatwić testowania rozwiązania, a także możliwości łatwiejszej translacji ruchów ludzkiego ramienia, zamiast fizycznego urządzenia wykorzystany zostanie model komputerowy wykonany w programie CAD/CAE. Model ten również powstanie w ramach tworzenia Pracy.

Praca ma charakter koncepcyjny i jej celem jest w równym stopniu stworzenie działającego prototypu rozwiązania, co nauka niezbędnych technologii i próba ich implementacji, a także eksperymentalne sprawdzenie możliwości wykorzystania tego typu kombinacji w praktyce inżynierskiej. W związku z tym nie jest narzucana końcowa precyzja, jaka jest niezbędna do uzyskania w toku prac. Dążone jednak będzie do uzyskania jak najbardziej zadowalających efektów. Na tej bazie będzie można w późniejszej perspektywie ustalić przydatność konsumenckiej kamery głębi oraz Naturalnych Interfejsów Użytkownika w miejscach, w których konieczne jest manualne sterowanie, nie ma jednak możliwości sterowania za pomocą GUI lub CLI, lub to sterowanie jest utrudnione.

[Type here]

[Type here]

[Type here]

[Type here]

Kinect i kamera głębi

Natural User Interfaces

Czym jest NUI

Zarówno w polskiej, jak i światowej literaturze nie pojawia się jak do tej pory jedna, uznana i stosowana definicja NUI. Posiłkując się literaturą zagraniczną wybrałem termin,

A natural user interface is a user interface designed to reuse existing skills for interacting directly with content.

który wydaje mi się najbardziej spójny oraz komplementarny:

Powyższa sentencja w dość ogólny sposób uzmysławia nam, z jaką kategorią interfejsów mamy do czynienia. Wraz z popularyzacją rozwiązań informatycznych a także wzrostem ich dostępności projektanci wymyślali kolejne „warstwy abstrakcji” które pozwalały na jeszcze bardziej wygodne sterowanie potężną mocą obliczeniową urządzeń. Ewolucje tą doskonale widać na bazie przejścia z interfejsów konsolowych (CLI) do graficznych (GUI). Zanim użytkownicy dostali w swoje ręce myszkę, pulpit oraz ikony rozwinęła się bardzo popularna w swojej epoce kategoria interfejsów, dziś znanych jako text-based interfaces. Można je uznać za swoisty etap przejściowy – mimo, że korzystały z rozwiązań technologicznych należących w bezpośredni sposób do CLI, to jednak przyświecała im ideologia zbliżona do tego co przyniesie ze sobą rewolucja GUI – przejście z zapytań typu bezpośrednio Input-Output którymi charakteryzowała się konsola na bardziej deskryptywny sposób interakcji z użytkownikiem.

Podobnie ma się sprawa z obecnie wprowadzanym NUI, które również w powolny sposób (za element przejściowy można tu uznać interfejsy telefonów – choć te w dalszym ciągu operują na abstrakcyjnych elementach typu ikony, można z nimi wchodzić w interakcje w sposób znacznie bardziej bezpośredni, niektóre interfejsy bywają też coraz mocniej skeumorficzne) zmieniają nasze podejście do obsługi, czy też coraz częściej niejako „współpracy” z elektroniką. Celem tego trendu jest zaadaptowanie jak największej ilości elementów z życia codziennego do obsługi sprzętu.

NUI rozwija się bardzo wielotorowo: za takowe uważa się w równym stopniu tak zwane Brain-Machine Interfaces rozpoznawanie mowy jak i gestów. W ramach tej pracy niejako wykorzystane zostaną właśnie te ostatnie.

[Type here]

[Type here]

[Type here]

[Type here]

Rozwój technologii NUI

Lata 70-dzisiaj - Eksperymenty Steve Manna

Steve Mann's "wearable computer" and "reality mediator" inventions of the 1970s have evolved into what looks like ordinary eyeglasses.



Steve Mann jest profesorem na Wydziale Nauki i Sztuki Uniwersytetu w Toronto i człowiekiem tytułującym się „Pierwszym cyborgiem świata”. Jako pierwsza na świecie osoba użył on określenia „Naturalne Interfejsy Użytkownika”, a do jego najciekawszych wynalazków zalicza się min. „ubieralny komputer”, z którego sukcesywnego ulepszania wraz rozwojem technologii słynie.

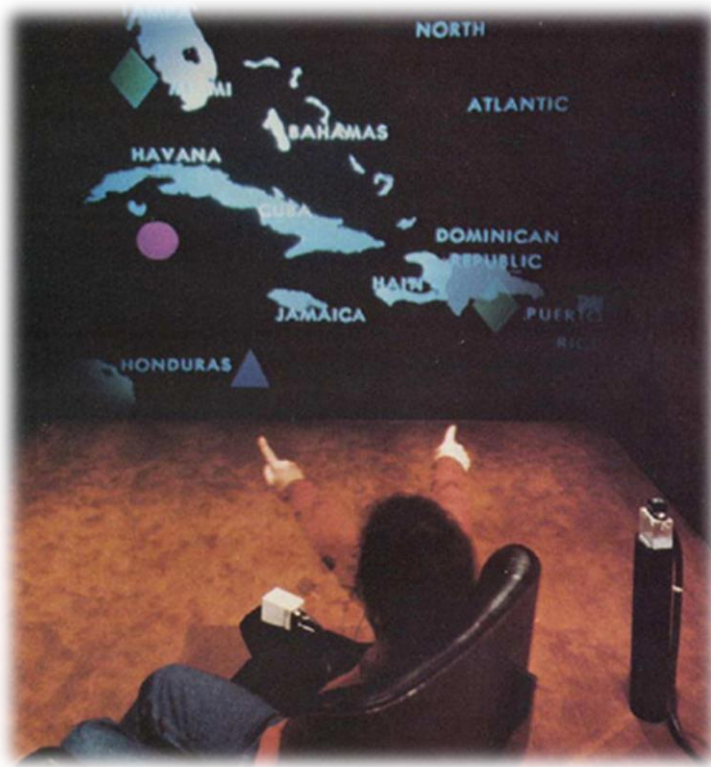
[Type here]

[Type here]

[Type here]

[Type here]

1980: Put-That-There, Richard A. Bolt, MIT



*Oficjalne zdjęcie pochodzące z pracy „Put-That-There”:
Voice and Gesture at the Graphics Interface” napisanej
przez Richard A. Bolta*

Put-That-There można uznać za pierwszą realną próbę stworzenia tego czym jest Kinect. Wprawdzie nie zastosowano tam kamery głębi, ale naukowcom rodem z MIT udało się stworzyć możliwość sterowania wirtualnymi obiektami zarówno za pomocą gestów jak i głosu, dzięki czemu stali się oni prekursorami tego typu rozwiązań. Instalacja znajdowała się w pokoju o wymiarach szesnaście na jedenaście stóp. Użytkownik siedział na krześle około 8 stóp od ekranu, na jednym z nadgarstków miał metalową kulę, zaś na głowę założony mikrofon. Pierwotna wersja pozwalała na prowadzenie statku przez Karaiby za pomocą ruchu ręki oraz ustawianie na mapie kolonialnych zabudowań. Projekt został w późniejszym okresie rozwinięty w inny, znany jako The Iconic System. Zamiast jednak magnetycznych kostek wykorzystane w nim zostały specjalne rękawice, zaś do dwóch poprzednich metod sterowania dodana została trzecia – śledzenie wzroku.

1998: DreamSpace, Marc Lucente, IBM

Kolejną ważnym eksperymentem było DreamSpace, znane także jako Vizualization Space, który podobnie jak Put-That-There wspierał równocześnie obsługę gestów (technologia IBM IntelliStation) i rozpoznawanie głosu (dzięki technologii IBM

[Type here]

[Type here]

[Type here]

[Type here]

ViaVoice). Chodził o wiele sprawniej od swojego poprzednika, dzięki wzrostowi mocy obliczeniowej sprzętu ze swojej epoki pozwalał na dodatkową manipulację w przestrzeni 3D. Nie wymagał on żadnego dodatkowego osprzętu w stylu rękawiczek rodem z Iconic System.

1999: Minority Report, John Underkoffler, MIT / Steven Spielberg



Film “Minority Report” Stevena Spielberga na podstawie opowiadania Philipe’a K. Dicka jest dziełem bardzo przełomowym dla twórców nowych interfejsów gdyż stanowił on pierwsze realne przedstawienie ich masowej publice i w znacznym stopniu przyspieszył pracę nad podobnymi rozwiązaniami. UI pokazane na potrzeby filmu nie było jedynie wymysłem artystów – do jego zaprojektowania zaproszono Johna Underkofflera, profesora MIT Media Labs, który podszedł do pracy od strony naukowej, dzięki czemu zaprezentowana wizja jest niezwykle realistyczna. Dzięki pracy nad filmem naukowiec zainicjował projekt Luminous Room, który został w 2010 roku zaprezentowany na konferencji TED. Twórcy Kinecta wskazują film jako bezpośrednią inspirację dla swojego urządzenia.

Historia powstania urządzenia Kinect

Technologia na której oparto kamerę głębi w urządzeniu została stworzona przez 4 inżynierów z izraelskiej firmy PrimeSense. Zeev Zalevsky, Alexander Shpunt, Aviad Maizels i Javier Garcia ogłosili ją w roku 2005 a całość została zawarta w patencie 20100201811. Została ona szybko przyuważona przez Microsoft, który licencjonował rozwiązanie oraz projekt chipsetu PS1080 na potrzeby użycia w swojej konsoli do gier.

Oprogramowaniem zajęła się firma RARE, zajmująca się wcześniej grami na konsole Nintendo, a zakupiona przez Microsoft w roku 2002.

[Type here]

[Type here]

[Type here]

[Type here]

Kinect został po raz pierwszy zapowiedziany 1 czerwca 2009, podczas targów E3 w Los Angeles, jako Project Natal. Nazwa Natal wywodzi się z brazylijskiego miasta o tej samej nazwie, a w wolnym tłumaczeniu z języka angielskiego oznacza „związany z narodzinami”. Nazwę końcową produktu, Kinect, ogłoszono 13 czerwca 2010, a samo urządzenie pojawiło się pół roku później, 4 listopada w Stanach Zjednoczonych a 10 listopada w Europie. W ciągu pierwszych 60 dni sprzedało się 8 milionów sztuk, a samo urządzenie trafiło do Księgi Guinnessa jako „Najszybciej sprzedająca się elektronika konsumencka w historii”. 9 stycznia 2012 roku podano, że ilość sprzedanych urządzeń przekroczyła 18 milionów.

Kinect początkowo nie posiadał sterowników przeznaczonych do pracy z PC. Firma Adafruit Industries zaoferowała nagrodę w wysokości 1000\$ dla osoby która jako pierwsza udostępni „Otwartoźródłowe sterowniki dla tego urządzenia. Mogą one chodzić pod dowolnym systemem, ale muszą posiadać dokumentację i otwartą licencję”¹. Nagroda wkrótce została podwyższona do 3000\$ i trafiła do rąk Hécтора Martína, który wydał OpenKinect, działający sterownik na system Linux. Dzięki temu urządzenie stało się obiektem zainteresowania hobbystów i inżynierów z całego świata.

Jeszcze w tym samym roku swój sterownik (a również framework programistyczny OpenNI) wydała firma PrimeSense. 17 czerwca 2011 Microsoft wydał swoje własne, oficjalne SDK w wersji Beta dostępne do użytku niekomercyjnego. Wersja 1.0 pozwalająca na użytek w projektach komercyjnych ujrzała światło dzienne 1 lutego 2012 wraz z informacją, że nad oprogramowaniem wykorzystującym Kinect pracuje 300 firm z całego świata. Kinect for Windows SDK oprócz samych sterowników posiada również zaawansowane API pozwalające na użycie dojrzałych algorytmów firmy Microsoft pozwalających na śledzenie ruchu i analizę głosu. Firma twierdzi, że są to dokładnie te same narzędzia, których sama używa podczas produkcji oprogramowania.

21 maja 2012 roku na rynek trafiła nowa wersja urządzenia, znaną jako Kinect for Windows, zawierająca taki sam hardware, jednak dzięki zmianom w firmware umożliwiono zmniejszenie odległości niezbędnej do poprawnego odczytu.

¹ <http://www.adafruit.com/blog/2010/11/04/the-open-kinect-project-the-ok-prize-get-1000-bounty-for-kinect-for-xbox-360-open-source-drivers/>

[Type here]

[Type here]

[Type here]

[Type here]

Wymagania sprzętowe

- Dwurdzeniowy procesor, 2.66-GHz lub szybszy
- Karta graficzna ze wsparciem możliwości Microsoft DirectX 9.0c
- 2 GB RAM (rekomendowane 4 GB)
- Osobny Hub USB

Urządzenie może być zainstalowane wyłącznie na systemach Windows 7 i Windows 8, porzucone zostało wsparcie dla starszych platform. Dodatkowo, sterowniki nie umożliwiają na pracę na Maszynie Wirtualnej ze względu na brak wsparcia sterowników dla wirtualizacji.

Budowa, parametry techniczne oraz sposób działania urządzenia Kinect for XBOX 360

Kinect for XBOX 360 jest urządzeniem należącym do kategorii określanej jako Kamery Głębi. W uproszczeniu można je zdefiniować jako kamery 3D dające obraz w taki sposób że do każdego piksela złapanego przez kamerę dodawana jest informacja o jego oddaleniu od urządzenia.

Na zestaw, z elektronicznego punktu widzenia, składają się:

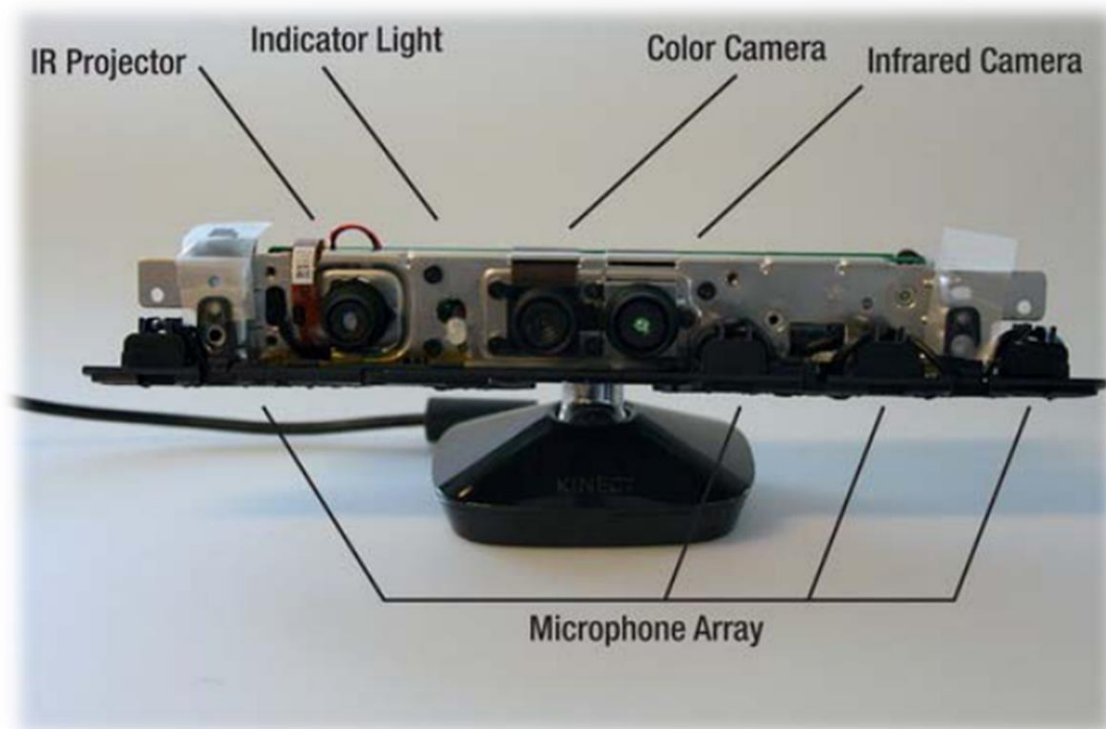
- Zestaw mikrofonów
- Emiter podczerwieni
- Odbiornik podczerwieni
- Kamera VGA
- Akcelatrometr
- Serwomotor

[Type here]

[Type here]

[Type here]

[Type here]



Sprzęt komunikuje się z komputerem za pomocą standardowego przewodu USB 2.0, wymaga również dodatkowego źródła zasilania.

Kinect for XBOX 360 niestety nie zawiera własnego wbudowanego procesora – jego jedyną namiastką jest procesor sygnałowy (DSP) służący do przetwarzania sygnału z macierzy mikrofonów. Wszystkie pozostałe obliczenia są wykonywane po stronie komputera PC.

Pionowy kąt widoczności to 57°, poziomy 43°. Serwomotor urządzenia umożliwia zmianę pionowego kąta ustawienia kamery o +/-28°.

Ze względu na ograniczoną przepustowość i związaną z nią koniecznością kompresji, kamera VGA pozwala na nagrywanie obrazu bez strat jakości w trzech formatach:

- 640 × 480 przy 30 klatkach na sekundę (frames per second - FPS) używając formatu RGB²
- 1280 × 960 przy 12 FPS używając formatu RGB
- 640 × 480 przy 15 FPS używając formatu YUV³

² The RGB format is a 32-bit format that uses a linear X8R8G8B8 formatted color in a standard RGB color space. (Each component can vary from 0 to 255, inclusively.)

³ The YUV format is a 16-bit, gamma-corrected linear UYUY-formatted color bitmap. Using the YUV format is efficient because it uses only 16 bits per pixel, whereas RGB uses 32 bits per pixel, so the driver needs to allocate less buffer memory.

[Type here]

[Type here]

[Type here]

[Type here]

Możliwe jest osiągnięcie większej ilości klatek na sekundę jednak wiąże się to z kompresją obrazu, stratą jego jakości oraz możliwymi wystąpieniami artefaktów/przekłamań.

Obraz w ramach kanału głębokości zawiera informacje o odległości pomiędzy kamerą a najbliższym do niej obiektem w danej jednostce powierzchni. Wspierane są poniższe rozdzielczości:

- 640×480
- 320×240
- 80×60

Wszystkie umożliwiają na odczyt przy 30 FPS.

System odczytu głębi nie opiera się na działaniu ultrasonaru lub radaru w celu oszacowania odległości, jak ma to miejsce w większości tego typu rozwiązań. Kinect został stworzony do użytku wewnętrznego, do pomieszczeń o ograniczonej objętości, powyższe metody nie pozwalają na wygodny precyzyjny odczyt w takim środowisku. Zamiast tego, Kinect posiada zdolność budowania „Mapy Głębi” znajdującego się przed nim obszaru. Dzieje się to poprzez użycie emitera podczerwieni generującej małe punkty o z góry zdefiniowanym, łatwym do odwzorowania i każdorazowego odtworzenia wzorcu pseudolosowych położzeń (znany jest on przez producenta jako *IR coding image*) oraz będącej w stanie je odczytać sensorze obrazu opartym na układzie CMOS opracowanym przez PrimeSense w tak zwanym „trybie nocnym”, wrażliwej na promieniowanie podczerwone. Posiada on specjalne filtrowanie, pozwalające na oddzielenie światła dziennego, dzięki temu nawet w jasnym środowisku jest w stanie dokonać odczytu. Dzięki temu, że Kinect wie, jakie powinno być ułożenie punktów, jest w stanie na bazie odległości między nimi oraz załamania wyliczyć dystans każdego jednego od kamery. Obliczenia te dokonywane są wewnątrz chipu PS1080 produkcji PrimeSense, przechowującego w pamięci wzorzec *IR coding image*. Tak powstała mapa głębi jest zapisywana jako 16 bit, z czego 13 (bity 3 do 15 licząc od zera) opisuje parametry obrazu, 3 pozostałe określają zaś przypisanego do niego użytkownika. Operacje na obrazie z kamery głębi dokonuje się jako bezpośrednią manipulację bitami. Pozwala to na znaczne przyspieszenie procesu, jest jednak dość kłopotliwe podczas pierwszego kontaktu ze sprzętem.

[Type here]

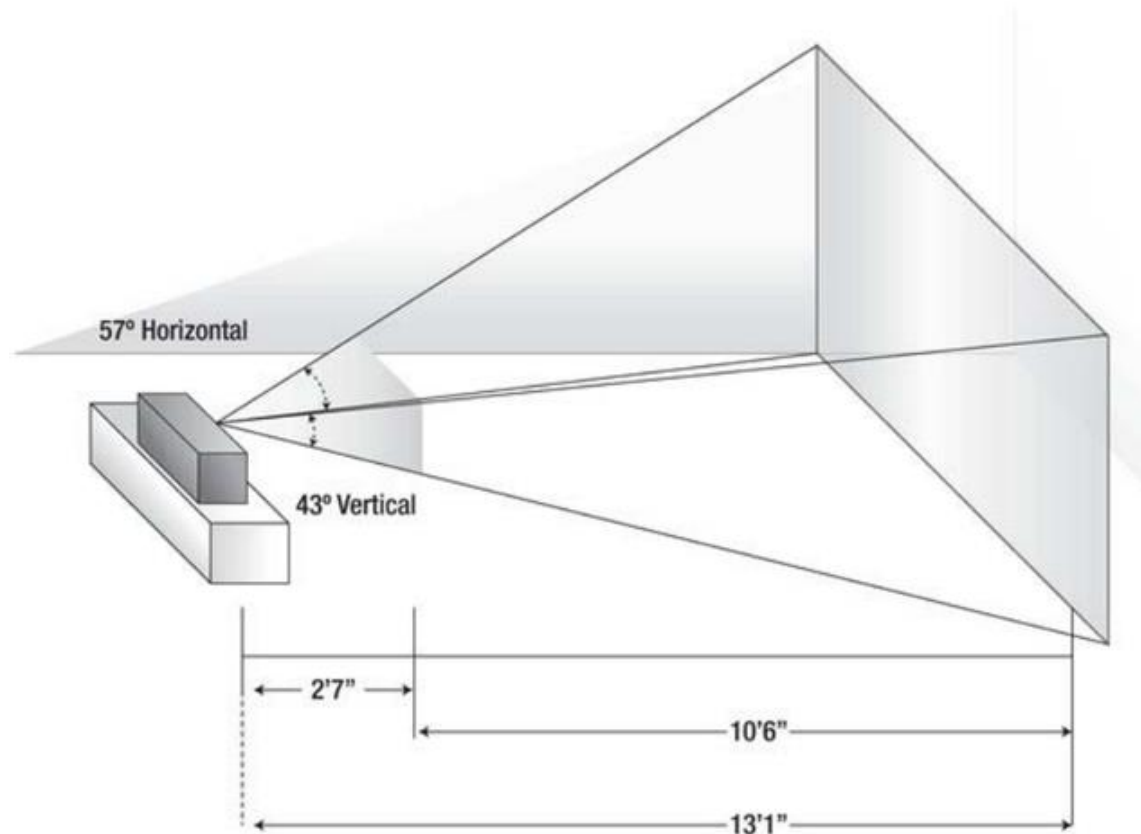
[Type here]

[Type here]

[Type here]

Optymalną jakość odczytu danych o odległości uzyskuje się w granicach od 1.2m do 4m, możliwe jest z poziomu oprogramowania wymuszenie zwiększenia tego zakresu ale wiąże się ono z brakiem wsparcia producenta.

Sensor powinien być trzymany z dala od bezpośredniego promieniowania słonecznego,



Pole części aktywnej podczas pracy urządzenia

umieszczenie urządzenia w bliskiej odległości od źródeł o dużym poziomie wydzielanego ciepła lub wibracji może zakłócić odczyt.

Porównanie dostępnych frameworków programistycznych

Jak wcześniej zostało wspomniane, Kinect spotkał się z entuzjastycznym przyjęciem programistów, którzy zauważyli potencjalne możliwości jego wykorzystania w ramach zastosowań nie związanych bezpośrednio z grami i konsolą Xbox. Jego możliwości i niski koszt zainteresowały zarówno producentów oprogramowania, marketingowców czy też pasjonatów robotyki. Jednak w celu rozpoczęcia jakichkolwiek prac inżynierskich potrzebny był sposób na proste odczytanie danych z poziomu samego urządzenia i udostępnienia ich zewnętrznym narzędziom pisanych przez osoby zewnętrzne. Dowodem na potrzebę pojawienia się na rynku takiego rozwiązania jest przypadek konkursu ogłoszonego przez Adafruit, który balansował na granicy legalności ze względu na

[Type here]

[Type here]

[Type here]

[Type here]

potrzebę zastosowania inżynierii wstecznej. Powstały w ten sposób OpenKinect ukazał się 10 Listopada 2011 roku i był wystarczającym narzędziem dla hobbystów i tak zwanych „Early Adopters” oraz ze względu na swoistą związaną z nim ideologię posiadał szereg zalet. Niestety, biblioteka nie jest aktualizowana od przeszło roku (ostatnie kosmetyczne poprawki przeszła przed 8 miesiącami). Obecnie znana jest pod nazwą libfreenect.

9 grudnia 2011 roku ukazuje się również otwartoźródłowy OpenNI Framework i mająca się nim opiekować OpenNI Organisation, która jest organizacją non-profit i została założona przez takie firmy jak twórca technologii kamery głębi PrimeSense, Willow Garage zajmujący się aplikacjami robotycznymi czy Asus, który wziął na siebie pomoc w rozwoju urządzeń wykorzystujących i promujących technologie stworzoną przez izraelskich inżynierów (owocem tego jest min. bliźniacza do Kinecta kamera o nazwie Xtion a także pierwsze próby stworzenia laptopów z odpowiednim wbudowanym rozwiązaniem sprzętowym). Ze względu na bardzo liberalną licencję i wsparcie PrimeSense zebrała wokół siebie dużą grupę entuzjastów. Pozwala na programowanie za pomocą C i C++. Dzięki wbudowaniu popularnej biblioteki computer vision OpenCV pozwala na sporą precyzję śledzenia pojedynczych palców, co pozwala na uzyskanie ciekawych efektów. Otwartość umożliwia również uruchomienie na dużej ilości platform, poza popularnymi systemami desktopowymi eksperymentalnie wspierane są również procesory ARM, co samo w sobie jest sporym atutem. Do prawidłowego działania sterownika potrzebne jest również tak zwane Middleware znane jako NITE.

Kolejną dużą premierą w tej gałęzi było Microsoft Kinect SDK for Windows. Premierę miało 16 czerwca 2011, kiedy to udostępniona była pierwsza wersja Beta. Jej potężną przewagą nad konkurencją były bardzo precyzyjne algorytmy śledzenia szkieletu, szybkość działania, dostęp do niskopoziomowych danych urządzenia a także wsparcie takich dodatkowych funkcjonalności jak rozpoznawanie mowy w języku angielskim (oraz inne skomplikowane algorytmy jej przetwarzania, jak usuwanie echa i szumu akustycznego), sterowanie serwo mechanizmem kamery. Pierwsze stabilne wydanie pojawiło się 21 maja 2012, wzbogaciła ona urządzenie o obsługę trybu near-mode, skeleton tracking w pozycji siedzącej, wsparcie dla nowych języków, obsługę wielu kamer, a także o nową licencję, umożliwiającą wykorzystanie komercyjne urządzenia. SDK jest ciągle aktualizowane, ostatnia poprawka z 8 października przyniosła wsparcie odczytu akcelerometru, lepsze opcje kalibracyjne i rozszerzenie odległości widzenia.

[Type here]

[Type here]

[Type here]

[Type here]

Kinect for Windows SDK można programować w języku C++, Visual Basic 6 oraz C# I dostępne jest tylko na platformie Windows. Komercyjne użycie jest możliwe tylko z kontrolerem Kinect for Windows, starszy Kinect for Xbox może być wykorzystywany tylko i wyłącznie w przypadku zainstalowania na dysku SDK zaś licencja dopuszcza jego użycie wyłącznie w celach testowych i edukacyjnych.

W międzyczasie firma Evoluce wydała swoje własne SDK, jednak nie przebiło się one do masowej świadomości, głównie ze względu na wysoką cenę i dość skomplikowane obwarowania licencyjne.

Poniżej zamieszczam porównanie Kinect for Windows SDK oraz OpenNI, ze względu na dojrzałość obu projektów, w celu stworzenia porównania, które ułatwi wybór odpowiedniego środowiska. Oba środowiska prezentuje w stanie na dzień 1 listopada 2012. Porównywać będę tylko i wyłącznie parametry możliwe do wykorzystania przy projektowaniu oprogramowania, w związku z czym pominę zaawansowane możliwości analizy dźwięku Kinect for Windows SDK.

	Kinect for Windows SDK	OpenNI

Wybrano Kinect for Windows SDK ze względu na poniższe założenia:

- Pod uwagę brane były tylko programy CAD/CAE posiadające wersje na platformę Windows, w związku z czym brak multiplatformowości nie był przeszkodą
- Kinect for Windows SDK umożliwia łatwą współpracę z innymi platformami Microsoftu
- Dobre wsparcie i jego gwarancja w przyszłości
- Dopracowane autorskie algorytmy śledzenia szkieletu
- Przyjazna licencja do celów edukacyjnych, akademickich

Skeleton Tracking

NUI API urządzenia używa kamery głębi w celu wykrycia obecności ludzi. Zostało ono zoptymalizowane pod kątem sylwetek odwróconych przodem do kamery, w związku z

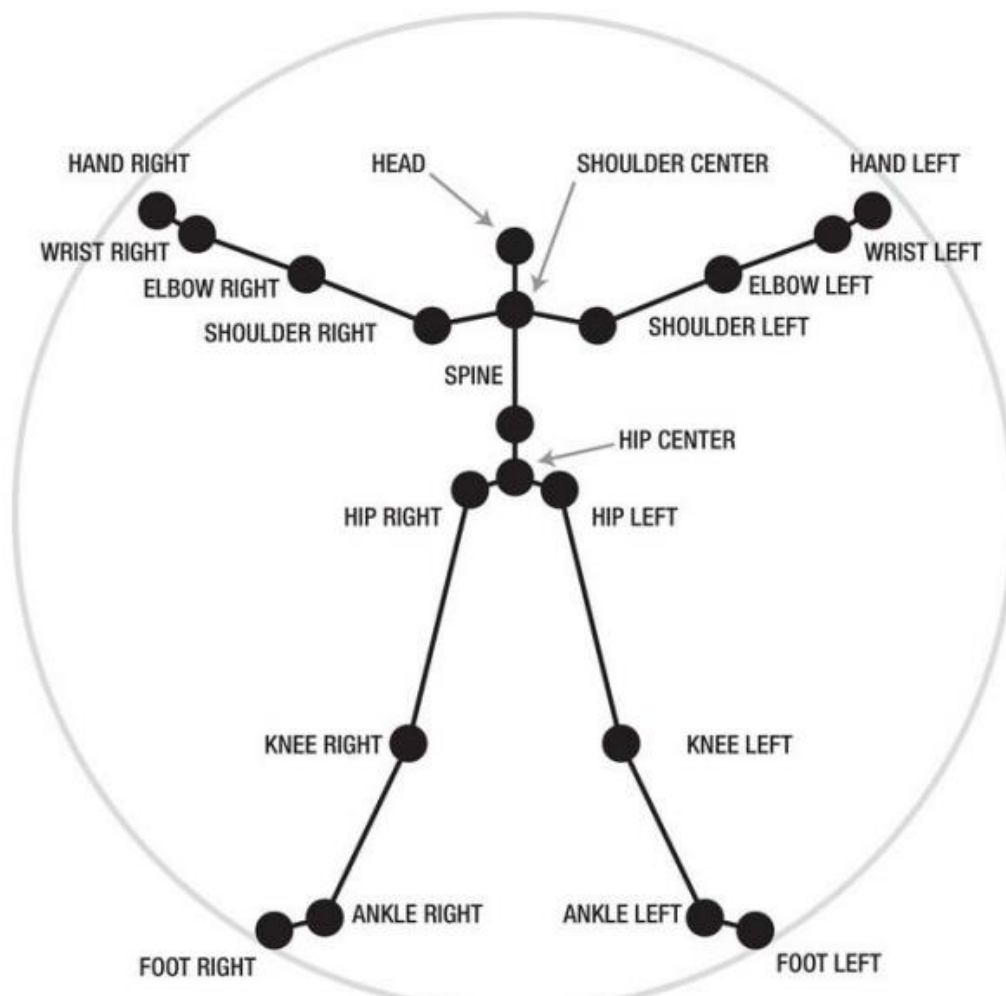
[Type here]

[Type here]

[Type here]

[Type here]

czym jeśli osoba jest ustawiona bokiem do czujnika odczyt może być przekłamany. O ile Kinect potrafi wykryć do sześciu ludzkich sylwetek, tylko dwie równocześnie mogą być trackowane. Dla każdej z nich NUI API stworzy kolekcje położeń charakterystycznych konkretnych 20 fragmentów ludzkiego ciała, każdy z nich jest opisany za pomocą współrzędnych kartezjańskich, ze środkiem układu ułożonym w miejscu położenia czujnika. W wypadku osób siedzących śledzone jest tylko 10 punktów



charakterystycznych zlokalizowanych w górnej partii ciała.

[Type here]

[Type here]

[Type here]

[Type here]

Licencja urządzenia Kinect for Windows SDK

Kinect SDK posiada licencje zezwalającą na użycie w projektach akademickich.

Microsoft® Kinect™ for Windows® Software Development Kit (SDK) from Microsoft Research (Non-commercial Use Only)

Punkt 2, ostatni akapit

For clarification, using the Software and Applications for personal experimentation is not a commercial purpose. It is also not a commercial purpose to use the Software in the process of teaching or academic research, even if you are regularly employed as a teacher or professor or if you intend to try to obtain research grants through such research.

W związku z powyższymi zapisami, możliwe jest wykorzystanie zarówno urządzenia Kinect for XBOX 360 jak i Microsoft Kinect for Windows SDK

Wybór programu CAD/CAE

Wybrany narzędziem, po wcześniejszym zapoznaniu się z oprogramowaniem konkurencji (*SolidWorks 2012 SP4*, *Catia ver. V6R2012x*) został program *Autodesk Inventor 2013/Build 2013*. Argumentami przemawiającymi za użyciem tego rozwiązania były:

- Znajomość programu, związana z uczestnictwem w zajęciach poświęconych jego obsłudze podczas toku studiów
- Wygodna licencja, pozwalająca na zastosowania akademickie w wygodny sposób, pozwalający na uniknięcie potrzeby podejmowania dodatkowych licencji w ramach uczelni oraz na pracę w domu
- Inventor 2013 API, pozwalające na wygodne sterowanie modelem z poziomu zewnętrznego oprogramowania napisanego za pomocą obiektowego języku programowania w natywny sposób (temat ten zostanie rozwinięty w kolejnych sekcjach)
- Możliwość wykorzystania licencji studenckiej przypisanej do konta students.autodesk.com. Licencja ta pozwala na użycie oprogramowania do celów edukacyjnych.

W pierwotnych założeniach, jeszcze na etapie pracy przejściowej, planowane było

- oparcie się na Java 3D API

[Type here]

[Type here]

[Type here]

[Type here]

- konwersji modeli z poziomu programu CAD/CAE do modelu 3D w formie graficznej
- późniejsza jego obsługa za pomocą Java 3D version 1.4 (interfejs Shape3D) oraz konwertera 3D Archive który pozwala na konwersję modelu do obiektu Java3D.

Podejście to wymagało długiego łańcucha konwersji (iam->3ds (format 3D Max)->obj (Java 3D)) co wiązało się z dużym narzutem czasowym, trudnością w utrzymaniu projektu (konieczność każdorazowego przejścia wskazanego procesu przy zmianach w modelu) a także niemożność zastosowania inżynierskiego podejścia, ze względu na działanie na modelu nieparametrycznym.

Podczas dodatkowych przeszukiwań dostępnych alternatyw okazało się, że tego typu założenia byłyby podejściem bardzo niepraktycznym. W związku z tym program Inventor firmy Autodesk posiadający Interfejs Programowania Aplikacji (API), który jak wspomniano umożliwia zewnętrznemu oprogramowaniu na dostęp do danych projektu (głównie złożenia/assembly) i manipulowania nim okazał się być znacznie lepszym rozwiązaniem.

Analiza Autodesk Inventor 2013 API

Definicja API

API (ang.) *Application Programming Interface*, Interfejs Programowania Aplikacji – jest terminem opisującym zestaw funkcjonalności wyprowadzonych na zewnątrz aplikacji pozwalającym oprogramowaniu tworzonemu przez osoby niezwiązane bezpośrednio z jej producentem na rozszerzanie jej możliwości. Dobrze skonstruowane, kompletne API daje możliwość wykorzystania takich samych lub zbliżonych możliwości z poziomu kodu źródłowego oprogramowania zewnętrznego, dzięki czemu programiści mają możliwość implementacji niestandardowych, niszowych funkcjonalności wykorzystując dane z aplikacji udostępniającej API lub zautomatyzowanie tych już istniejących w celu wyeliminowania powtarzalności ich wykonywania, dzięki czemu oprogramowanie staje się bardziej spersonalizowane, a praca bardziej produktywna, wygodniejsza i przyjemniejsza.

[Type here]

[Type here]

[Type here]

[Type here]

Dzięki API istnieje możliwość lepszego wdrożenia oprogramowania w ramach już istniejącego procesu produkcyjnego i usprawnienie jego możliwości współpracy z pozostałymi niezbędnymi aplikacjami.

Sposób działania Inventor 2013 API

Inventor API wykorzystuje technologie firmy **Microsoft** znanej jako **Automation** (wcześniej określanej jako **OLE Automation**). Umożliwia ona komunikację międzyprocesową i jest oparta na standardzie **COM** (Component Object Model), definiującym sposoby tworzenia interfejsów programistycznych na poziomie binarnym, na bazie którego powstały również niskopoziomowe API samej firmy Microsoft. Technologia Automation jest szeroko używana w ramach różnorodnego oprogramowania dostępnego na system Windows w różnych wersjach. Komunikacja odbywa się jako połączenie klient-serwer. Dzięki niemu aplikacja zewnętrzna określana jako *Automation Controller* (Kontroler Automatyzacji) ma możliwość dostępu oraz manipulacji danymi udostępnionymi przez *Automation Objects* (Obiekty Automatyzacji), jak określamy oprogramowania których możliwości chcemy rozszerzyć. Zaletą tego systemu jest jego duża uniwersalność jeśli chodzi o ilość wspieranych środowisk oraz języków programowania. Za wadę można uznać ograniczenie obsługiwanej platformy wyłącznie do systemów firmy Microsoft.

Dostęp do Inventor API może być uzyskany na bazie czterech różnych metod: poprzez Makra (inaczej określane jako obiekty Visual Basic for Application), Dodatki, tak zwane Apprentice Servers, pozwalające na dostęp do dokumentów stworzonych w oprogramowaniu firmy Autodesk przez zewnętrzne oprogramowanie oraz .NET Framework przy użyciu *Primary Interop Assemblies*, które umożliwiają wywołaniu kodu niezarządzanego z poziomu kodu zarządzanego. Plik PIA zawiera oficjalny opis niezarządzanych typów, które zostały zdefiniowane przez ich wydawcę i udostępnia kompatybilne z .NET Framework interfejsy COM Automation API. W ramach tej pracy wykorzystana zostanie właśnie ta ostatnia metoda.

PIA występuje już w ramach zainstalowanego oprogramowania Inventor 2013, jednak jest ono niedostępne dla programisty. W celu jego wykorzystania niezbędne jest doinstalowanie *DeveloperTools.msi*. W folderze instalacyjnym pakietu można znaleźć plik Autodesk.Inventor.Interop.dll, którego dołączenie wystarczy do rozpoczęcia pisania

[Type here]

[Type here]

[Type here]

[Type here]

zewnętrznego oprogramowania mającego możliwość współpracy z programem Autodesk Inventor 2013.

Dobór języka programowania i środowiska wykorzystanego podczas tworzenia oprogramowania na potrzebę pracy.

Inventor API w wersji 2013 pozwala na łatwe wykorzystanie czterech języków programowania: C++, Delphi, C# oraz Visual Basic.

Z dostępnych języków w Inventor API wybrany został język C#, wybór ten wiąże się z:

- Obiektością języka
- Dużą dojrzałością i bibliotekom zewnętrznym
- Nowoczesnym podejściem do oprogramowania
- Wygodnemu tworzeniu interfejsów użytkownika przy użyciu Windows Presentation Foundation (WPF)
- Frameworkowi .NET przyspieszającemu pracę programisty
- Wspólna składnia z Kinect for Windows SDK
- Wsparcie komercyjne dużej korporacji jest atutem przy późniejszych próbach biznesowego wdrożenia rozwiązań

Za środowisko pracy wybrano Visual Studio Professional 2012, ze względu na wygodę tworzenia oprogramowania i zaawansowane możliwości. Licencja edukacyjna uzyskana w ramach programu Microsoft Dreamspark przeznaczonego dla studentów uczelni technicznych.

Wdrożenie pomysłu

Model Inventor API

Opis Modelu

Wykorzystany model został stworzony na potrzeby pracy w oprogramowaniu Autodesk Inventor 2013. Jego celem nie jest wierne stworzenie realistycznego projektu robotycznego ramienia. Ma on posłużyć jako próba schematycznego oddania zależności pomiędzy poszczególnymi połączeniami kostnymi ręki. Nie przewidziano w ramach jego tworzenia możliwości jakiegokolwiek wykorzystania praktycznego. Ze względu na duże wykorzystanie mocy procesora przez kamerę głębi a także potrzebę jego animowania w

[Type here]

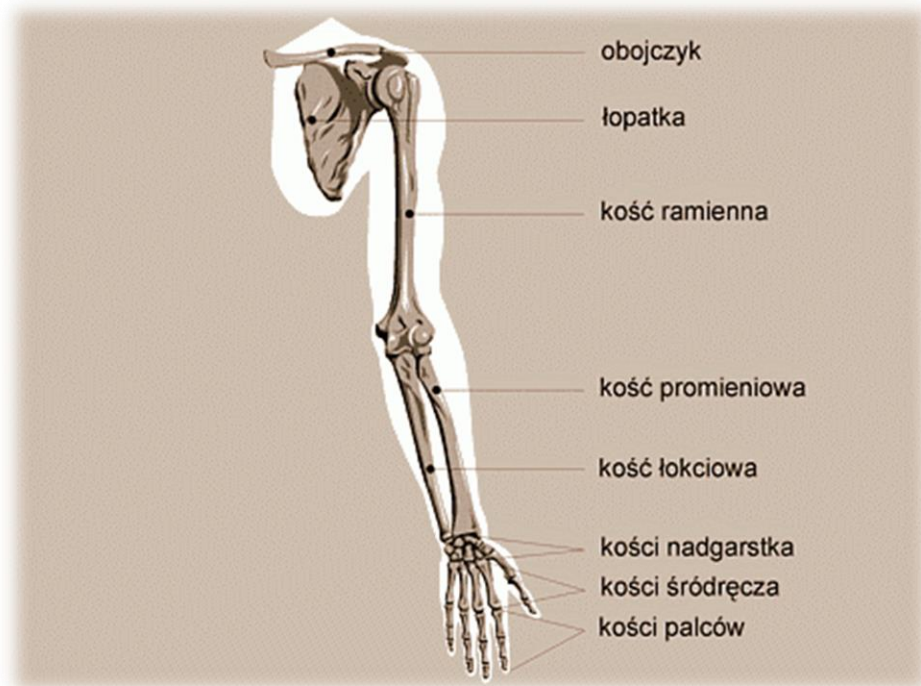
[Type here]

[Type here]

[Type here]

czasie rzeczywistym, postawiono na ograniczenie niepotrzebnych elementów do niezbędnego minimum. Ze względu na koncepcyjny charakter pracy, ograniczono się do modelu 2D przyjmując obrót kości w stawach wyłącznie w jednej osi.

Model składa się z czterech elementów połączonych wiązaniami typu „Kąt” i „Wstaw”. Każde z nich zostało ograniczone w sposób programowy – nie wykorzystano „Zestawów Kontaktowych” wykrywających kolizje, ponownie ze względu na duże wymagania mocy komputera przez część aplikacji związaną z przetwarzaniem danych odczytanych z kamery. Wiązania typu „Wstaw” pozwalają na uzyskaniu stałej osi obrotu poszczególnych elementów, wiązanie typu „Kąt” wiąże ze sobą geometrie obrotu poszczególnych elementów względem siebie. Ustawienie wiązań „Kąt” wszystkich elementów na wartość zero sprawia, że ich krawędzie w poszczególnych osiach są do siebie wzajemnie równoległe.



Poszczególne elementy odpowiadają odpowiednio za:

- Obojczyk – Element1
- Kość ramienia – Element2
- Kość promieniową – Element3
- Kości nadgarstka – Element 3

Każdy element ma wyłącznie jeden stopień swobody – dookoła osi równoległej do osi Z układu bazowego.

[Type here]

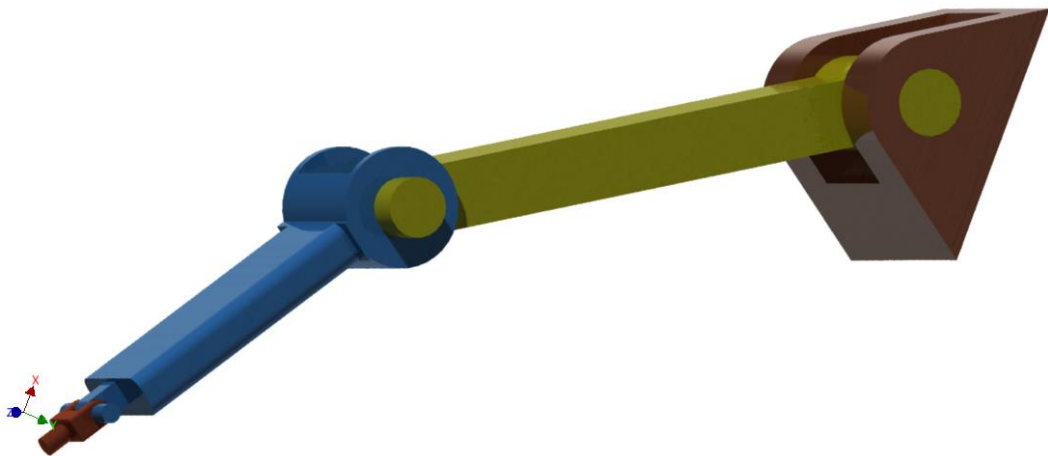
[Type here]

[Type here]

[Type here]

Poszczególne wiązania w mają na celu oddanie w uproszczeniu i wyłącznie w wybranych płaszczyznach stawu barkowego, łokciowego, a także sposobu działania kości nadgarstka zrzutowanych na wybraną płaszczyznę prostopadłą do osi Z układu bazowego.

Do modelu nie są przypisane prawdziwe materiały, nie posiada także żadnych właściwości dynamicznych. W celu polepszenia klatkażu pracy wyświetlania sugerowana jest jak najbardziej uproszczona reprezentacja.



Geometria z punktu widzenia Inventor API

Złożenia nie posiadają ściśle zdefiniowanych współrzędnych położenia poszczególnym części. Opierają się one na koncepcie tzw. „Chwilowej Geometrii”. Jest to sposób tworzenia „Chwilowych Obiektów”, które są niewidocznymi, matematycznymi reprezentacjami bez odwzorowania w komponentach graficznych, takich elementów jak punkty, macierze czy wektory. Służą one do tworzenia punktów odniesienia do manipulacji poszczególnych elementów API.

W odróżnieniu od swoich graficznych odpowiedników, obiekty „Chwilowej Geometrii” nie zawierają się w granicach graficznego obiektu. Prosta jest nieskończona, płaszczyzny nie są ograniczone.

Na potrzeby pracy inżynierskiej wykorzystana w pewnym stopniu obiekt Macierz do pozycjonowania elementu Obojczyka. Pozwala on na precyjne manipulowanie obiektem w przestrzeni 3D. Umożliwia sterowanie zarówno w przesunięciem, jak i obrotem, dzięki korzystaniu z macierzy kwadratowej, w której wykorzystywane są wyłącznie trzy pierwsze rzędy. Każdej macierzy można przypisać system współrzędnych początkowych,

[Type here]

[Type here]

[Type here]

[Type here]

dzieje się to poprzez polecenie *SetCoordinateSystem*. Macierz transformacji można wypełnić zarówno ręcznie, jak i dzięki wsparciu funkcji pomocniczych, takich jak *TransformBy* czy *SetToRotateTo*. Spis wszystkich funkcji można znaleźć w dokumentacji API Inventor 2013.

Budowa strukturalna złożenia

Każde złożenie programu Inventor 2013 jest reprezentowane po stronie API w ramach obiektu *AssemblyDocument*. Jest to programowalny nadzbiór zawierający wszystkie informacje dotyczące złożenia. Pozwala on, poprzez *AssemblyComponentDefinition* na wyekstrahowanie poszczególnych jego współrzędnych. Każda wstawiona część jest reprezentowana w pamięci jednokrotnie i określana jako Occurence, zawierające informacje dotyczące elementu, a także pozwalające uzyskać informacje o danej części niedostępne bezpośrednio z poziomu złożenia dzięki conceptowi Proxy. Jest on również niezbędny w momencie, kiedy pojawia się więcej niż jedno wystąpienie danego elementu w ramach złożenia.

Proxy jest referencją do obiektu poprzez poszczególne Occurence i zamiast poszczególnego wystąpienia zwraca on natywny obiekt. Pozwala w bardziej świadomy sposób identyfikować z poziomu API poszczególne parametry i zwracać je w kontekście pożądanym przez użytkownika końcowego. Są one wykorzystywane również w ramach interfejsu Inventor API, mając na celu ukryć implementacje danej części od jej poszczególnego użycia. Dzięki temu wszystkie manipulacje z poziomu złożenia nie mają wpływu na element bazowy. Tak naprawdę złożenia nie przechowują żadnych prawdziwych geometrii, a jedynie Proxy do nich, a wszelkie transformacje odbywają się na bazie umiejscowienia danej części w ramach złożenia i tylko w kontekście złożenia. Każdy element dostępny z poziomu złożenia ma przypisaną mu klasę reprezentującą jego Proxy, zarówno całe Subzłożenia, Occurency jak i ich krawędzie lub powierzchnie. W momencie kiedy użytkownik wybiera w programie jakikolwiek element złożenia, może być pewien że tak naprawdę manipuluje on jego Proxy.

Proxy są tworzone za pomocą metody *CreateGeometryProxy*. Każdy Obiekt typu Proxy zawiera dokładnie te same metody, co prawdziwy element.

[Type here]

[Type here]

[Type here]

[Type here]

Kinect SDK – sposób odczytu danych szkieletowych

Wybór części do trackingu

W związku z faktem, że do pracy wymagane jest niemal współbieżne działanie Inventor 2013 oraz Kinect SDK, a także w celu wyeliminowania możliwości popełnienia błędu, zdecydowano się ustawić domyślnie pewne parametry i ograniczenia:

- Jako osoba śledzona przypisana jest pierwsza osoba, która pojawiła się w widoku kamery
- Domyślnie śledzona jest lewa ręka danej osoby

Aplikacja jest sparametryzowana, a część domyślnych ustawień można zmienić z poziomu karty „Opcje” w aplikacji.

Pobieranie danych

Każdy człon w ramach śledzenia szkieletu charakteryzuje się trzema współrzędnymi zapisanych w systemie kartezjańskim: X, Y oraz Z. Punkt początkowy układu to wejście to położenie czujnika podczerwieni pochodzą z jednego z trzech strumieni danych przekazywanych przez urządzenie, a znanego jako *SkeletonStream* i można je uzyskać na dwa różne sposoby – poprzez zdarzenia (Events) oraz odpytywanie (Pooling). Obie metody przebiegają w nieco inny sposób.

Pobieranie poprzez zdarzenia jest metodą dostępną tylko przy użyciu Windows Presentation Foundation. Wprowadziła ona model wiązania poszczególnych metod wewnątrz programu z konkretnymi z konkretnymi zdarzeniami (Programowanie Event-Driven), będącymi tu „nadawcami” poprzez metodę zbliżoną do wzorca projektowego znanego w literaturze jako „Obserwator”. Metoda klasy subskrybuje się do konkretnego zdarzenia i zostaje wywołana każdorazowo wraz z wykonaniem danej akcji (wciśnięcie przycisku, wpisanie tekstu), w wypadku Kinecta powiązana akcja jest odpalana każdorazowo, gdy urządzenie może przekazać kolejny pakiet danych do aplikacji, opierając się na zdarzeniu *FrameReady*. Poza każdorazowym sprawdzeniem, czy Event nie jest pusty, nie wymaga on większego zaangażowania od programisty.

Pooling działa w odmienny sposób. Każdy zestaw danych Kinecta posiada metodę o nazwie *OpenNextFrame*. Każdorazowo kiedy aplikacja odpytuje o kolejną „klatkę”, ustalany jest czas oczekiwania na odpowiedź, tak zwany „timeout”, i program stara się uzyskać dane do czasu jego upływu. Jeśli ten nastąpi a akcja nie zostanie wykonana,

[Type here]

[Type here]

[Type here]

[Type here]

zwracany jest pusty obiekt. Możliwe jest wystąpienie trzech różnych sytuacji, które powodują wystąpienie wyjątku *InvalidOperationException*: gdy urządzenie jest niedostępne, gdy strumień danych jest niedostępny lub gdy dane już zostały pobrane przez zdarzenie – jednorazowych danych nie można pobrać dwa razy przez dwa różne wątki.

Zaletą Poolingu jest jego nieco większa wydajność w stosunku do sterowania zdarzeniami, w związku z faktem, że z tymi drugimi związana jest duża ilość operacji wspierających. Dodatkowo, Pooling jest wymagany w momencie, kiedy WPF nie jest wykorzystywany – przykładowo podczas projektowania aplikacji konsolowej lub gry korzystającej z frameworka XNA. Wiąże się on jednak z bardziej skomplikowanym kodem źródłowymi i większym tzw. „Boilerplate” aplikacji.

W ramach pracy inżynierskiej wykorzystany został Pooling ze względu na możliwość odcięcia aplikacji od jej interfejsu graficznego, co pozwala na stworzenie zewnętrznego API do dowolnego użytku, jako że do sterowania modelem nie jest potrzebny interfejs i w aplikacji ma charakter dodatkowy. Dodatkową zaletą jest też większa wydajność tego rozwiązania.

Pobieranie danych zostało przerzucone z wątku UI do osobnego, dodatkowego wątku, w celu zwiększenia responsywności interfejsu.

Jednostka w której odczytywane są dane szkieletu to metry.

Część Aplikacyjna

Architektura klasy

Sterowanie aplikacją

W aplikacji zastosowane zostały

Klasa odpowiadająca za odczyt danych z Kinecta

Klasa odpowiadająca za sterowanie programem Inventor 2013 poprzez API

Integracja z Inventor 2013

W celu ułatwienia korzystania z aplikacji stworzono prosty Plugin (sprawdzono kompatybilność z Inventor 2013), który dodaje do wstążki Inventor 2013 skrót umożliwiający szybkie uruchomienie. Aplikacja pojawia się w ramach zakładki „Złóż”,

[Type here]

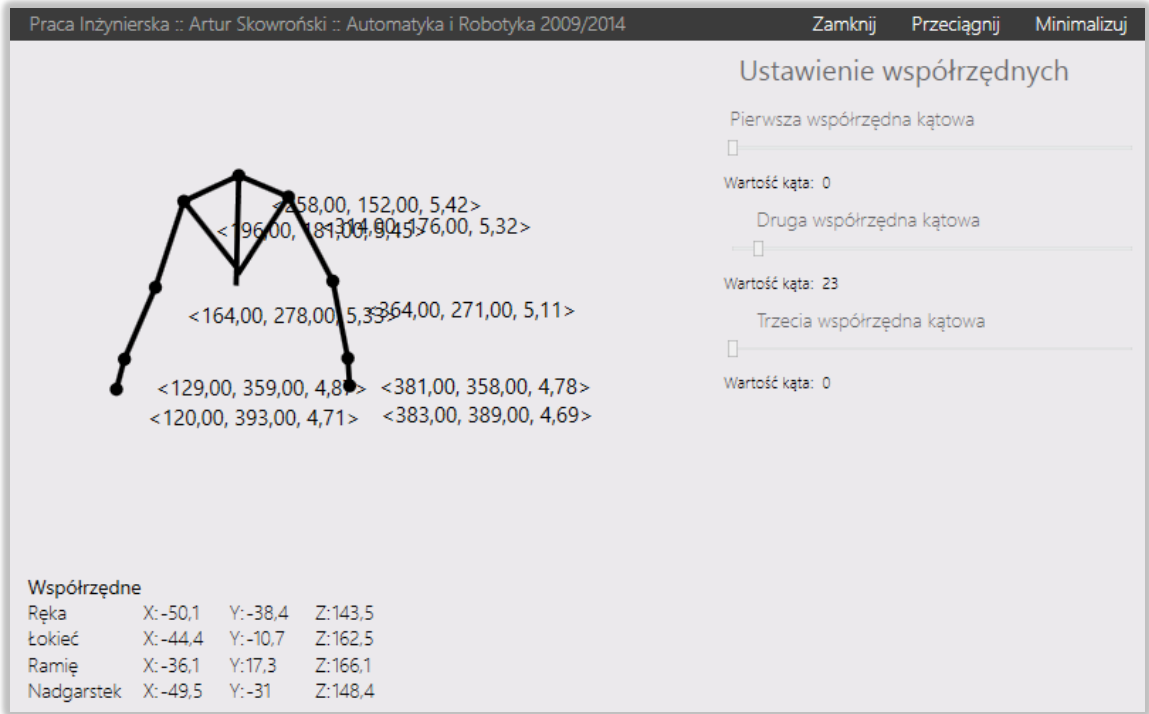
[Type here]

[Type here]

[Type here]

w ramach panelu „Praca Inżynierska”. Ikona skrótu została stworzona zgodnie z wytycznymi Autodesk Inventor Icon Guide.

Interfejs Użytkownika



[Type here]

[Type here]

[Type here]